

mcp-ratchet — Tool Qualification Plan

Which DO-330 process areas apply at TQL-5, the artifact that satisfies each, the correctness fix the plan commits to, and how the tool and its baseline are held under configuration management.

Doc ID: MR-TQ-003 Rev: 2026-08-29 TQL: 5 Qualification basis: Criteria 3 (MR-TQ-001 §5)

Read Order: 3 of 5

A Tool Qualification Plan is the tool-side equivalent of a PSAC: it states what is being qualified, to what level, against what requirements, and how each applicable DO-330 objective will be met and by which artifact. This plan is written for the drift-detection function of mcp-ratchet at TQL-5. It commits to closing the one correctness gap MR-TQ-002 found, and defines the configuration management the qualification depends on.

§1 Scope of this plan and what it does not reproduce

- Tool and function:** mcp-ratchet, runtime tool-surface drift-detection function only, per MR-TQ-001 §6. The prompt-injection check is excluded (TOR-11).
- Level:** TQL-5, from Criteria 3 at any airborne software level (MR-TQ-001 §5).
- Requirements:** TOR-1 through TOR-11 in MR-TQ-002.
- Not reproduced:** DO-330's numbered objective tables and its per-TQL applicability matrix are paywalled RTCA content. This plan maps DO-330's process areas *by intent* to artifacts in this set. A real program's TQP cites the actual objective identifiers from the standard; this study cannot, and says so rather than inventing them.

§2 DO-330 process areas mapped to artifacts

DO-330 organises tool qualification into the same shape as DO-178C software work: planning, development, verification, configuration management, quality assurance, and certification liaison. At TQL-5 the development-side depth is minimal — the emphasis is on demonstrating the tool meets its operational requirements in its operational environment. The mapping this plan commits to:

DO-330 PROCESS AREA (BY INTENT)	HOW IT IS MET HERE	ARTIFACT
Tool planning	This document: scope, level, basis, objective mapping, CM, QA, acceptance.	MR-TQ-003
Tool operational requirements	Falsifiable "shall" statements stating the correspondence the tool's output must have to the activity it supports.	MR-TQ-002
Tool operational verification — requirements-based testing in the operational environment	A verification case per TOR, executed on the qualified environment (MR-TQ-002 §2), with recorded pass/fail and the evidence trail. This is the bulk of a TQL-5 effort.	MR-TQ-004
Tool configuration management	§6 of this plan: the qualified commit, the baseline file, and the artifact index, each identified and change-controlled.	MR-TQ-003 §6
Tool quality assurance	§7: independent review of the TORs and verification results against this plan; the honest-gap discipline carried through every document.	MR-TQ-003 §7
Tool qualification liaison — accomplishment summary	The closing document: what was done, what each TOR's verification concluded, residual limitations, and the verdict.	MR-TQ-005

§3 Why landing at TQL-5 is the point, not a shortcut

TQL-1 through TQL-4 add tool low-level requirements, tool architecture and design, and structural coverage of the tool's own code — the development-side objectives that make qualifying a complex tool expensive. Criteria 3 avoids all of that. The drift-detection function earns Criteria 3 honestly: it is an additional automated control, every alert is reviewed by a human, and no manual process is removed on the strength of its output (MR-TQ-001 §5). The narrow scope and the retained human review are what make a requirements-based demonstration in the operational environment a sufficient and affordable qualification. Widen the usage and the level, the cost, and the objective set all go up together.

§4 Correctness fix this plan commits to — TOR-9

MR-TQ-002 §4 found one requirement the current source does not meet: a malformed, unreadable, or unsupported-version baseline file raises an unhandled exception in `load_baseline` (`proxy/run_proxy.py`), and there is no `fingerprint_schema_version` gate. The plan is:

Fix 1 — guarded load — Wrap the read and parse in `load_baseline` so that `json.JSONDecodeError`, a missing required key, and `OSError` are caught and converted into a typed `BaselineError` result distinct from both "baseline present and valid" and "no baseline file".

Fix 2 — schema-version gate — In the loader, compare the file's `fingerprint_schema_version` against the running tool's `FINGERPRINT_SCHEMA_VERSION`. A mismatch is a `BaselineError`, not a best-effort parse.

Fix 3 — fail-safe in the proxy — `server_side.py`'s `on_list_tools` already handles `baseline is None` by logging a `no_baseline` record and not claiming "no drift". Add a parallel `baseline_error` path with its own `error_type`, so a reader can tell "never had a baseline" from "the baseline was broken". Under either condition the proxy must not proceed such that tool calls run with drift silently un-evaluated.

SEQUENCING

This is real code work on the mcp-ratchet repository. It is done before MR-TQ-004's TOR-9 verification case, so that case runs against fixed behaviour and passes as qualification evidence rather than being logged as an open defect.

§5 Report markers — TOR-8 and TOR-11

REQUIREMENT	MARKER COMMITTED
TOR-8 — missing baseline must not read as clean	Any report or dashboard view produced with no baseline carries a visible "DRIFT NOT EVALUATED — no baseline" banner. The machine-readable output sets <code>drift_evaluation: "not_performed"</code> rather than omitting drift fields, so absence of a drift list cannot be read as an empty (clean) drift list.
TOR-11 — injection check carries no credit	In any report used for compliance, the qualified drift-detection result sits in its own labelled section. The prompt-injection section carries a "NOT A QUALIFIED RESULT" marker. The two are never merged into one verdict.

§6 Tool configuration management

ITEM	IDENTIFICATION	CHANGE CONTROL
Qualified tool	mcp-ratchet at the exact git commit SHA recorded in MR-TQ-005, with <code>fingerprint_schema_version = 1</code> .	Any commit change voids the qualification until re-verified. The qualified commit is tagged in the repository.
Baseline file	<code>baselines/<slug>.json</code> for the credited assistant, plus a recorded SHA-256 of that file.	Placed under the program's CM. <code>scanner.run_scan</code> will not overwrite an existing baseline, so a re-baseline is a deliberate, separately authorised act — recorded with who, when, and why.
Qualification data	MR-TQ-001 through MR-TQ-005 and MR-TQ-004's verification evidence, each with a revision date.	Revisions are additive; a superseded revision is retained. The <code>Read Order: N of M</code> tag is updated across the set if a document is inserted.

§7 Tool quality assurance and the independence limitation

QA activity for this qualification: an independent reviewer checks that the TORs in MR-TQ-002 are complete against the reliance claim, that every TOR has a verification case in MR-TQ-004, that each case's recorded result matches its stated verdict, and that the CM items in §6 are identified and controlled.

INDEPENDENCE LIMITATION — CARRIED TO MR-TQ-005

In this study the tool author, the person writing the qualification, and the notional applicant are the same person. DO-330 expects the tool's qualification verification to be independent of its development. A real program using this material must have that review performed by someone other than the tool's author. This limitation is stated here and repeated in the accomplishment summary; it is not waved away.

§8 Acceptance

On a real program, this plan is accepted by the applicant's certification authority before verification begins, and the qualified tool's use is agreed between the applicant and the tool developer. For this study, "acceptance" means: the plan is complete, its objective mapping is defensible, and the verification in MR-TQ-004 is executed exactly as scoped here — with the open items in MR-TQ-002 §4 and the independence limitation in §7 above resolved or explicitly accepted before the verdict in MR-TQ-005.

§9 Carried forward to MR-TQ-004

- One verification case per TOR, run on the qualified environment, each with inputs, expected result, recorded result, and evidence reference.
- Re-run the existing `tests/test_fingerprint.py` and `tests/test_drift.py` as evidence for TOR-1 through TOR-7, and add the cases they do not cover: `tool_removed` , `schema_changed` , `annotations_changed` , and a modified tool where several fields change at once.
- Execute the TOR-9 verification case only after Fixes 1–3 are merged; record the commit that closed it.
- Verify the TOR-8 and TOR-11 report markers on a real generated report, not just in the template source.